

VNUHCM - University of Science
Faculty of Information Technology

CS486 – Lecture 10

Indexing

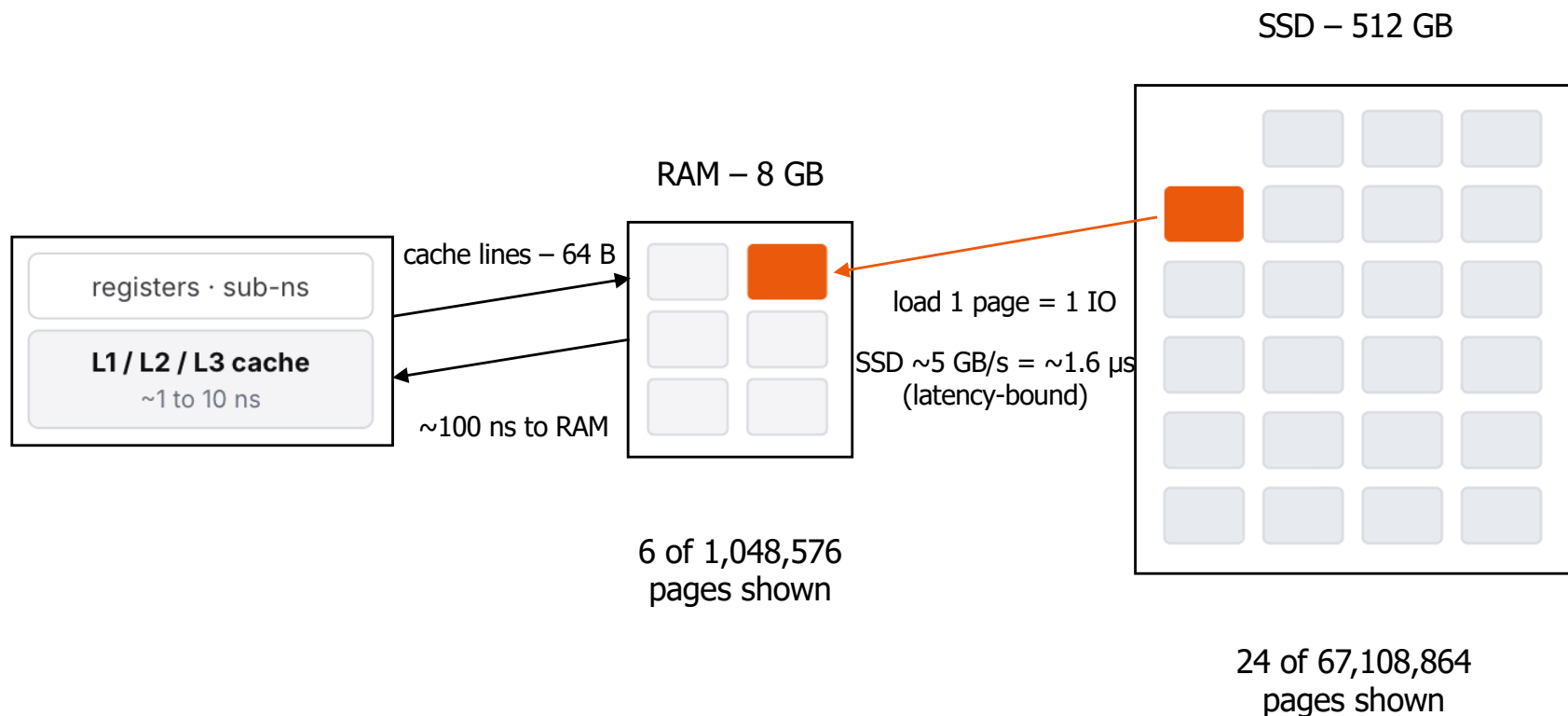
Le Thi Nhan
lt Nhan@fit.hcmus.edu.vn

Content

- **Storage**
- **Indexing**
 - Hashing
 - B+ Tree
- **Ref**
 - Stanford's CS145

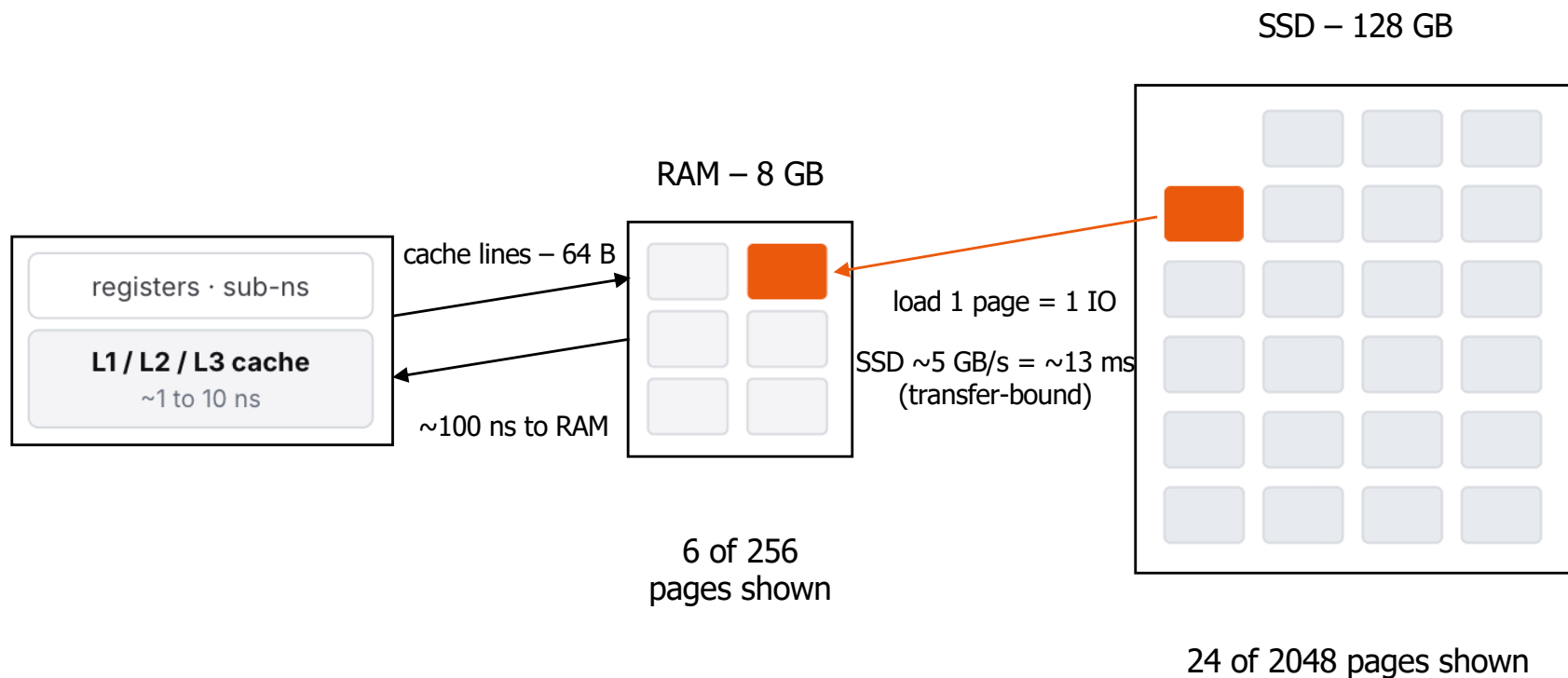
Storage

- Pages move from SSD to RAM one at a time
 - One page (8 KB) = one IO



Storage

- Pages move from SSD to RAM one at a time
 - One page (64 MB) = one IO



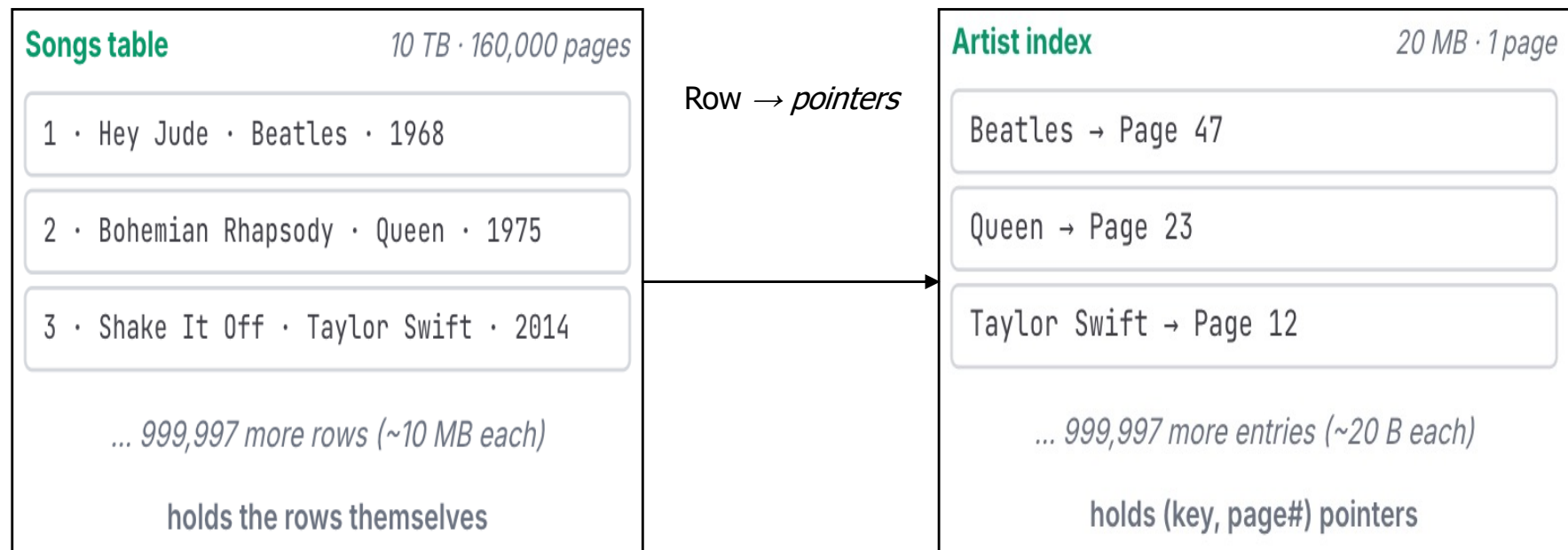
Index

- An index is an auxiliary data structure that maps a key to the page on where matching rows live
- Users (user_id, name, email)
Songs (song_id, title, **artist**, genre) : artist = 'Beatles'
Listens (listen_id, user_id, song_id, rating, listen_time)
 - 160,000 pages (10TB) (1 page = 64 MB)
 - Without index : scan all 160,000 pages
 - With an index on `artist` : 1 page
 - Looks up "Beatles lives on page 47"
 - Reads page 47
 - Returns the matching songs

What an index stores

- Data & index live on disk in the **64 MB** pages
- Data page holds whole rows
 - A song with its audio and metadata : 10 MB
 - 1M songs : 10 TB
- Index page holds `(search-key, page#)` pairs
 - `(search-key: 4 bytes) + (page#: 4 bytes) = 8 bytes`
 - `(search-key: 12 bytes) + (page#: 8 bytes) = 20 bytes`
 - 1M pairs : 20 MB

What an index stores



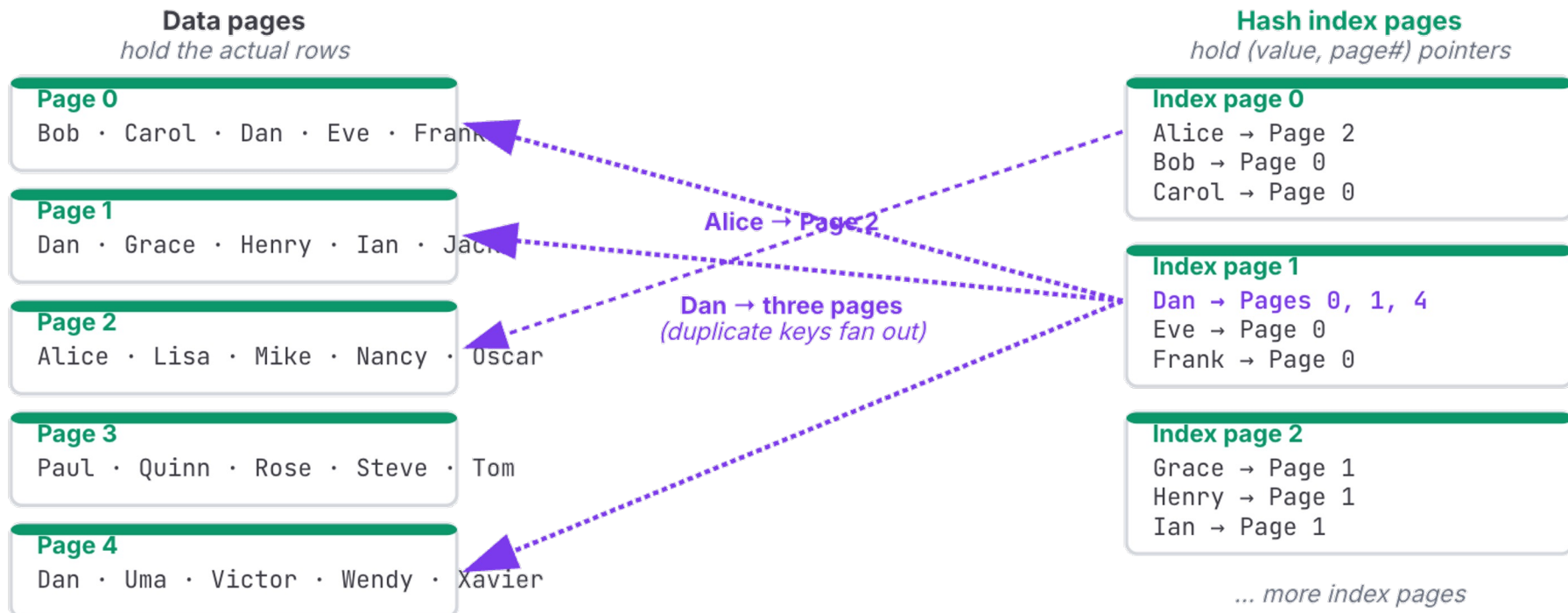
Tradeoff

- Faster reads in exchange for extra storage
- Slower writes for index maintenance
- Indexes should only be created on columns that are *frequently used* by queries for searching, filtering, or joining, and which are highly ***selectivity***
 - Each value in that column returns only a small number of records
 - Query performance >> the cost of maintaining the index

Hash Index

- Maps a key to its page via a hash function
- Support equality lookups
 - WHERE user_id=1 : return the page in one IO
- But no range scans
 - WHERE 10<user_id<20 : give no help
 - Because neighbour keys land on random pages

Example



Lookup(x): hash(x) → index page → find entry → read data page

Organizing a Hash Index

- Basic: one giant dictionary

```
index = {  
    "Alice": 2,  
    "Bob": 0,  
    "Carol": 0,  
    "Dan": [0, 1, 4],  
    # ... a billion more entries  
}
```

```
page_num = index["Alice"]
```

- Works fine until the dictionary stops fitting in RAM
- One billion entries at 16 bytes each is 16 GB

Organizing a Hash Index

- Partitioned: split across N pages (default)

```
# split into N=256 smaller indexes
index_pages = [ {...}, {...}, ... ] # 256 pages

def lookup(key):
    which = hash(key) % 256           # pick the page
    return index_pages[which][key]
```

- Each index page is about 64 MB and fits comfortably
- The extra hash step is free, and the semantics are unchanged

Organizing a Hash Index

- Partitioned: split across N pages (default)

```
# split into N=256 smaller indexes
index_pages = [ {...}, {...}, ... ] # 256 pages

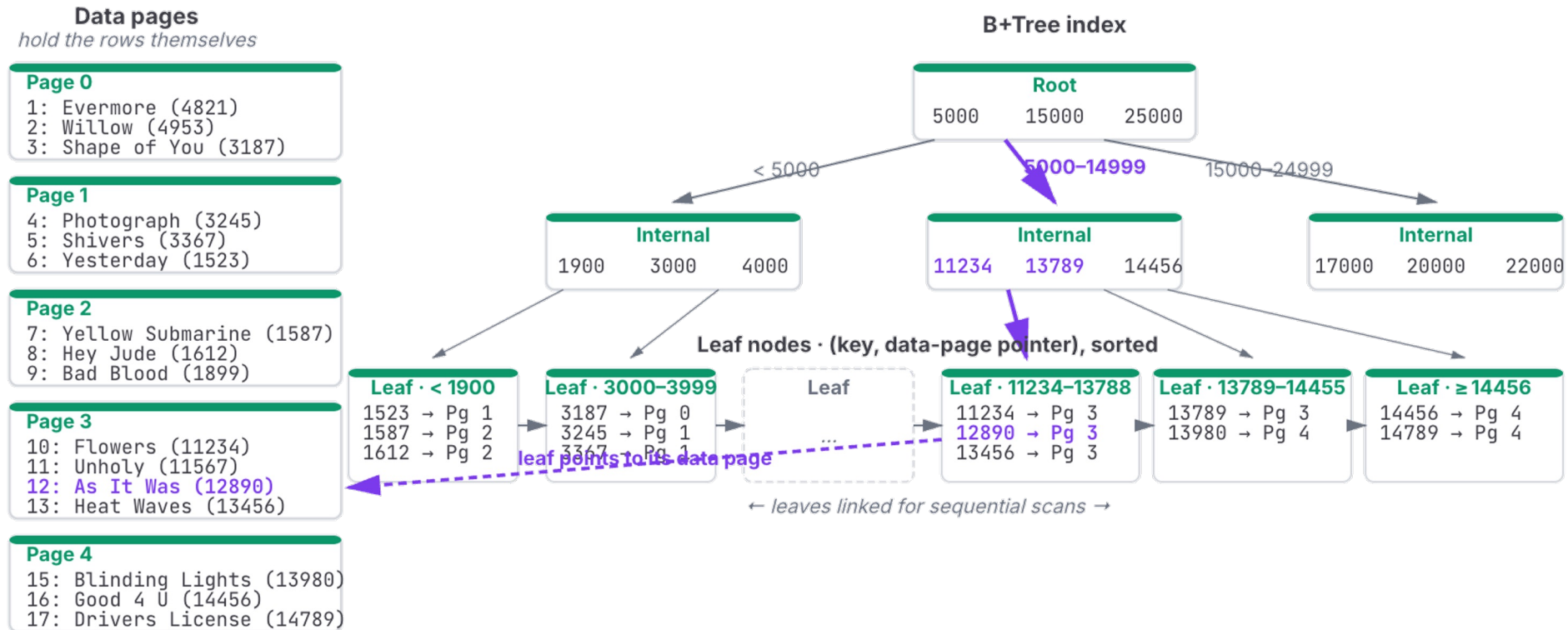
def lookup(key):
    which = hash(key) % 256           # pick the page
    return index_pages[which][key]
```

- Hash choice : important
 - A poor hash will cluster keys into a few buckets and degrades lookup to $O(N)$
 - Well-studied hashes (MurmurHash, xxHash...)

B+ Tree Index

- Keep keys sorted in a balanced tree
 - Support lookups and range scans by walking the leaf pages in order
 - `user_id = 42`
 - `4.0 < rating < 5.0`
 - `name LIKE 'Ali%'`
 - But do not help
 - Use functions on indexed columns : `lower(name)='dan'`
 - Suffix matches : `name LIKE '%son'`
 - Non-equality : `age <> 30`
- because the tree cannot narrow the search space

Example



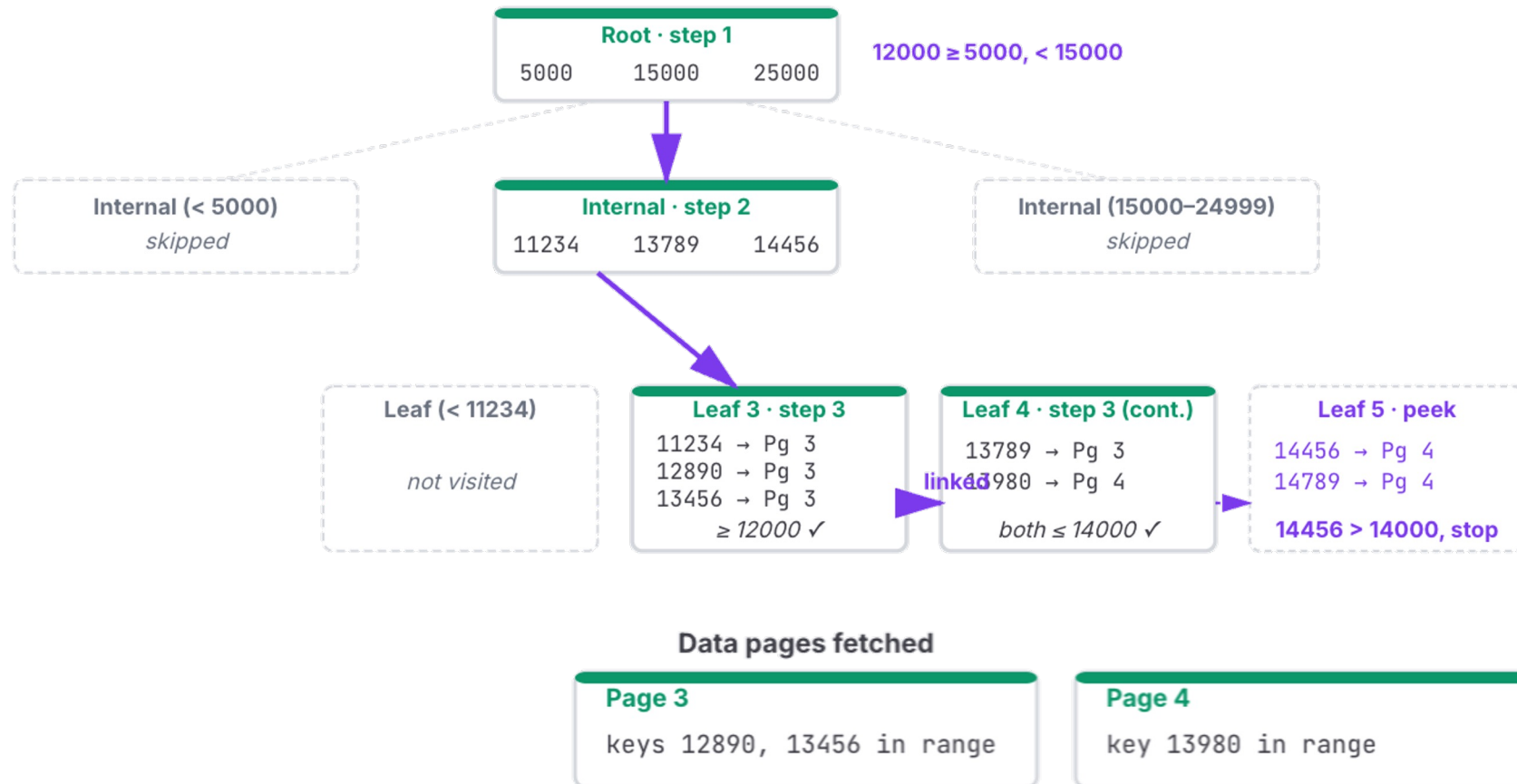
Fanout : 4 (3 keys plus 4 child pointers per node)

B+ Tree Height

- Fanout in real DB
 - Thousands to millions depending on page size
- Eg.
 - A billion-key table (10^9) with fanout about 1,000 (10^3)
 - $Tree\ height = \log_{Fanout}(N) = \log_{10^3}(10^9) = 3$
- Every layer is 1 IO + 1 IO to read the data page
- A point lookup is 4 IOs regardless of table size

Example

Walking the range $12000 \leq \text{key} \leq 14000$



Cost

- 5 index IOs (root, internal, leaves 3 + 4, peek 5)
- 2 data-page IOs (pages 3 and 4)
- 7 IOs total vs. 160,000 full scan $\rightarrow \sim 23,000 \times$ faster

Shallow Tree

- Fanout
 - How many children an internal node has
 - How many `(key, pointer)` pairs fit on one page
- High fanout = a shallow tree = few IOs per lookup

$$\text{Fanout} = \left\lfloor \frac{\text{page size}}{(\text{key size} + \text{pointer size})} \right\rfloor$$

$$\text{Height} = \log_{\text{Fanout}}(\text{number of keys})$$

- B+ Tree lookups feel constant-time even as tables grow

Hash vs. B+ Tree

	Hash Index	B+ Tree Index
Lookup	$O(1)$ equality, hash to bucket, read the entry	$O(\log N)$ any key, descend root $\rightarrow$ internal $\rightarrow$ leaf
Range / ORDER BY	None, hash scrambles neighbours	Native, leaves are linked left-to-right
Write cost	Update one bucket (split on overflow)	Random write + occasional rebalance
Typical use	Primary keys, caches	General-purpose OLTP
Production examples	Redis, Memcached	PostgreSQL, MySQL, InnoDB

What if the data is sorted?

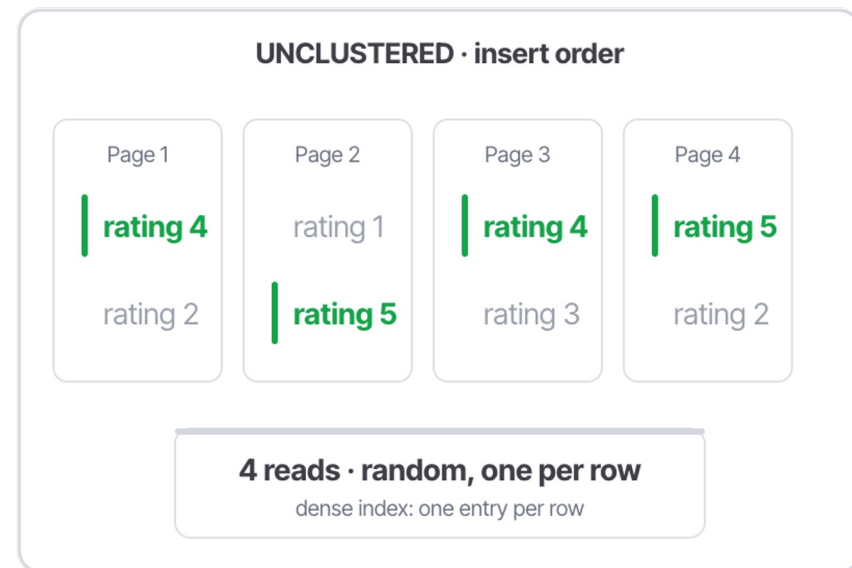
- A dictionary

- $n = 171,476$ words spread over $N = 1,000$ pages

	Sorted	Unsorted
Word order	Alphabetical, from "aardvark" to "zyzzzyva"	Scattered randomly across pages
Index density	Sparse: one entry per page (first word + page number)	Dense: one entry per word
Lookup	Find "coffee" with one index lookup ("clutch-coffin to page 217"), jump there, scan a few rows	"coffee" is on page 847, "coffeehouse" on page 12: every word needs its own pointer
Index size	~1,000 entries (N pages)	~171,476 entries (n words, 170x larger)

What if the data is sorted?

- DB table (**clustering**)
 - Query : Find every rating of 4 or 5



Sorting is what shrinks both the reads and the index

What if the data is sorted?

■ Indexing

	Clustered Index	Unclustered Index
Physical layout	Table data is physically sorted by the index key	Table data is in insert order, not sorted by the index key
Index density	Sparse (one entry per page)	Dense (one entry per row)
Range scans	Sequential reads: find the first page, walk forward	One IO per matching row, jumping around the table
Bottom line	Best for the access pattern you scan most	All non-primary indexes are usually unclustered

